

Appendix: Measurement Methodology

Reproduction, evidence scope, and the limits of the measurements

Matan Cohen · Yuval Kogan | Technion | HWSW Final Project

A1. Timing and provenance

The headline results use the course QEMU/KVM VM, Ubuntu 22.04, release CPython 3.10.12 and pyperformance 1.14.0. Each JSON contains 120 measured values (40 value-bearing worker runs, three values each); calibration is separate. Reported uncertainty is sample standard deviation across those values, not a confidence interval. Use the full-precision JSON means to calculate ratios, rather than rounded display values.

Comparison	Files in the results directory
Nbody stock / Python	baseline_nbody.json / optimized_nbody.json
Pyflate stock / Python	baseline_pyflate.json / optimized_pyflate.json
Nbody Python / native	fallback_nbody.json / native_nbody.json
Pyflate Python / native	fallback_pyflate.json / native_pyflate.json

The stock/optimized and fallback/native comparisons were separate experiments. Older stock/optimized JSONs lack backend metadata; the later fallback/native files explicitly record python and native. The pure-Python means agree closely across the two experiments, but this agreement does not retroactively add missing metadata.

Development evidence is labeled separately. Pyflate's ablation and cProfile tables come from Windows / CPython 3.12.6. Nbody's detailed unrolling sweep and pyflate's older 25× native kernel example come from WSL2 / CPython 3.10.21. Different machines, interpreters and estimators prevent using these absolute timings as VM results. Likewise, a paired median speedup need not equal the ratio of two independently reported minimum times.

Reproducing the comparisons

From the repository root, compare saved data without changing it:

```
python3 -m pyperf compare_to results/baseline_nbody.json \
    results/optimized_nbody.json
python3 -m pyperf compare_to results/fallback_pyflate.json \
    results/native_pyflate.json
```

For fresh native/Python comparisons, use the same interpreter with the wheels installed:

```
HWSW_BACKEND=python python3 benchmarks/bm_nbody/run_benchmark.py \
    --rigorous --inherit-env HWSW_BACKEND -o /tmp/nbody-python.json
HWSW_BACKEND=python python3 benchmarks/bm_nbody/run_benchmark.py \
    --rigorous --inherit-env HWSW_BACKEND -o /tmp/nbody-native.json
```

Use new output names for each experiment. Repeat with bm_pyflate for decompression. pyperformance run creates its own environment; direct benchmark execution avoids assuming that environment contains the extension wheels. Native mode fails when a wheel is unavailable. The runner scripts at the repository root provide the full stock/optimized workflow.

Correctness

Nbody comparisons check all state components and energy after the same initial conditions and 20,000 steps. Exact == is the observed result for the shipped Python and native kernels; a tolerance pass alone does not establish equality, and numerical equality is not a general byte-representation test (for example, signed zeros). Pyflate checks the decoded bytes against bz2.decompress, the original MD5, the intermediate L-vector and the ending bit position.

A2. Phase counters: evidence and limits

dev/pyflate/phase_cpi.py repeats one phase after preparing its input. The internal wall timer covers the repeated phase; perf stat wraps the **entire process**, including imports and input preparation. The values below reproduce results/pyflate_phase_cpi.txt; wall times are divided by repeat count and rounded. These are phase timing and aggregate-counter measurements, not a CPI stack decomposing execution into stall causes.

Phase	Repeats	ms/iter	IPC	Cache misses / refs
Native decode	300	3.304	1.49	1.284%
Whole inverse BWT	20	137.720	2.92	2.601%
Chase loop	30	47.820	2.41	1.831%
RLE4	100	23.836	2.89	1.623%

IPC uses instructions:u / cycles:u; the generic cache events were collected without that user-only modifier. Ratios are process-wide. Cache events do not identify which access missed, the latency it incurred, or how much useful work overlapped it. Therefore we do not infer an exact DRAM-access interval, or assign all cycles per traversal step to interpreter overhead.

The chase loop performs `end = T[end]`, reads `L[end]`, writes an output byte and runs Python loop machinery. Its setup builds `T` outside the wall timer but inside the counter interval. The `L`-vector contains 336,184 bytes; `T` is a Python list of indices, whose list slots and integer objects add memory. The final 399,360-byte output size is not the traversal table size. Whole BWT includes substantial table-construction work as well as the dependent traversal.

PMU and profiling configuration

- **Counting:** the VM experiments observed usable cycle/instruction counts and zeros when too many hardware events were requested together. Scripts split events into passes of at most four. Check actual counts and enabled/running time rather than assuming multiplexing.
- **Sampling:** tested `cycles -F` invocations produced zero samples, whereas fixed-period `cycles -c` and software `cpu-clock` sampling worked. The experiment demonstrates a working configuration; it does not establish the kernel's internal cause or a universal KVM limitation. Core and reference cycles are distinct events even when counts are close.
- **Call chains:** `--call-graph dwarf,16384` produced usable debug-build stacks where frame-pointer unwinding failed. Debug-build profiles describe that build's costs and are not used as release timings. Python-frame sampling uses `py-spy` separately.
- **Sample rate:** check `perf_event_max_sample_rate` after system tuning. A rate of 1 Hz was observed in the tuned setup and suppresses useful profiles. Do not assume it persists after a reboot or is the default on another machine.

Example counter command (whole-process scope):

```
HWSW_BACKEND=native perf stat \
  -e cycles:u,instructions:u,cache-references,cache-misses -- \
  python3 dev/pyflate/phase_cpi.py --phase chase --repeat 30
```

To establish a native BWT memory bottleneck, measure a native implementation, isolate or subtract setup with a justified method, and sweep working-set size. The current counters are insufficient for a measured SRAM-versus-DRAM accelerator comparison.

A3. Readable profiles and review checks

The main reports use print views derived from the saved, startup-filtered py-spy SVGs. `report/make_figures.py` reconstructs their call trees from rectangle nesting, merges sibling frames with the same function name across source lines, and draws the benchmark subtree. It conserves each selected subtree's sample counts, checks child-count bounds, and records the selected root and sample count in `report/fig/profile_counts.json`.

Grouping removes source-line detail; selecting a benchmark root removes harness context. Widths are normalized to that selected root and show **inclusive** counts. A child's samples are included in its parent; widths on different rows must not be added. Small boxes may omit labels, with full titles retained in the SVG. The original `results/pyspy_*.svg` files remain the source evidence. These figures summarize a few hundred samples and should not be treated as precise estimates of small differences.

Earlier flame-graph trimming

The committed py-spy views exclude startup samples; the original `*_full.svg` files preserve them. The C-frame figures in `results/flame_*.svg` also elide leading context and cap depth. Recorded caps are depth 9 / 4.5% affected samples for stock nbody and depth 36 / 4.4% for stock pyflate. Capping loses leaf detail. Removing samples can change the normalization, and merging frames can change individual box widths, even when surviving sample counts are preserved. The old claim that every displayed width stays unchanged was too broad.

The C-level profiles remain available as supporting evidence: `results/flame_nbody_stock_full.svg`, `flame_nbody_opt_full.svg`, `flame_pyflate_stock_full.svg` and `flame_pyflate_opt_full.svg`. Flat self-time output is in the corresponding `perf_report_*.txt` files. Timings and headline speedups come from JSON measurements, not from graphical widths.

Review spot checks, 2026-09-07

Short checks were rerun on the course VM using the repository's existing scripts. SHA-256 hashes matched the local copies for both shipped benchmark files and both `rs_check.py` scripts. The following timings are **minimums of three interleaved rounds**, not new rigorous benchmark results:

Comparison	Python	Native / hybrid
Nbody stock integration	231.552 ms	9.458 ms
Pyflate T3 symbol loop	115.873 ms	3.201 ms
Pyflate T3 complete decode	278.244 ms	161.072 ms

Nbody's shipped Python kernel and its Rust kernel matched stock state and energy exactly after 20,000 steps. Pyflate's Python and hybrid outputs matched bz2; the L-vector and ending bit position matched too. These checks corroborate functionality and approximate performance, but do not replace the saved 120-value comparisons.

The remote working tree contained existing changes, and its saved `baseline_pyflate.json` was missing. No synchronization or replacement of remote results was performed. The local baseline JSON remains the source for the report's stock comparison.

Source references

- Python: [self versus cumulative profile time](#).
- Python: [bytecode is version-specific](#).
- Reproduction scripts and raw data: [project repository](#).
- The two benchmark PDFs contain workload results and hardware boundaries; their matching `report_*.txt` files are generated text companions. Build with `report/build.ps1` on Windows or `report/build.sh` on Linux/WSL.